FINAL PHASE REPORT

QUANTALYTICS

Architecture, Analytics & FinOps

From IBM Quantum and NOAA telemetry to a Gold dimensional model and Power BI.

PROJECT

Data Analysis and Visualisation | Semester Project

REPOSITORY

github.com/HassanNawaz14/Quantalytics

PRODUCTION DATE

25 October 2026

Evidence-based report. Open verification items are disclosed; the Spark UI DAG capture and actual cloud billing are not supplied.

01 Executive summary

Scope, delivered architecture, evidence and the remaining submission dependencies.

Quantalytics studies IBM quantum processor reliability (T1, T2 and readout error) against NOAA geomagnetic Kp and F10.7 solar-flux signals. It implements a Databricks/PySpark/Delta Lake medallion pipeline and a dimensional Gold layer consumed by a Power BI semantic model. The analysis is exploratory: temporal association is not evidence of causation.

GOLD FACTS	GOLD DIMENSIONS	POWER BI MODEL	REPORT PAGES
5	10	19	12
Atomic, daily, audit, quality	Conformed descriptive tables	Explicit core relationships	5 original + 7 appended

What is established by the repository

- **Implemented source code:** IBM/NOAA collectors, Bronze and Silver notebooks, Gold notebook with full/incremental logic and OPTIMIZE ZORDER calls.
- **Dimensional model:** five fact tables, ten dimensions, one shallow snowflake through **dim_qubit → dim_backend**; logical keys and joins defined in DBML.
- **Power BI-native project: dashboard/Quantalytics_pbip_v2/** contains Databricks connection parameters, semantic model, DAX and 12 report pages.
- **Scheduling definitions:** weekly collector workflows and two three-task Databricks job specifications, each Bronze → Silver → Gold.

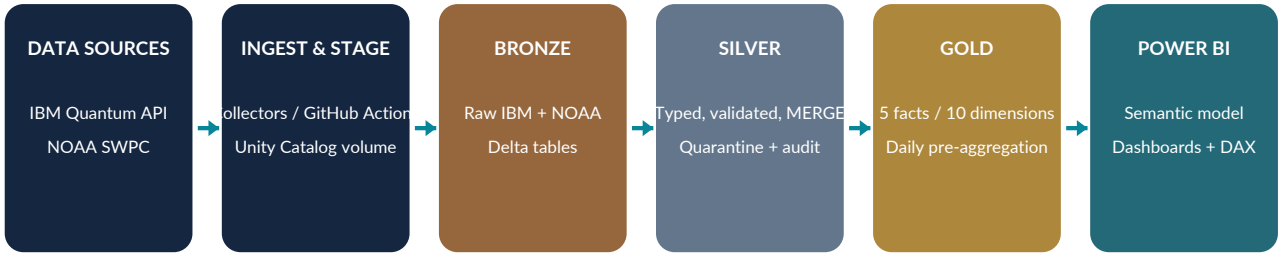
Evidence limitations that must not be hidden

> The ZIP does **not** contain a real Spark UI DAG capture, a billing/DBU export, or saved outputs proving a recent Gold execution. Job definitions contain blank **existing_cluster_id** values; actual deployed orchestration cannot be established from these templates alone.

Repository basis: 78b8d4d ("Adding Dash", 25 Oct 2026). Observational examples elsewhere in this report use five Gold CSV exports shared separately by the team on 24 Oct; they are not all included inside the Git ZIP.

02 End-to-end architecture

System boundary: collector workflows, Databricks Bronze/Silver/Gold, and Power BI.



Orchestration & observability

GitHub scheduled collection	Databricks jobs	Audit + quarantine
Supported by workflow YAML	Bronze > Silver > Gold dependencies def	Events and invalid-source records

Diagram depicts designed flow. Deployed job dependencies are not verified by repository JSON alone.

Operational sequence

Step	Inputs → outputs	Engineering mechanism
01 Collect	IBM Quantum API / NOAA SWPC → staging files	GitHub Actions scheduled/manual collectors; Databricks CLI uploads
02 Bronze	CSV/JSONL → raw Delta	Explicit StructType; source_file lineage; load timestamps
03 Silver	Bronze Delta → typed Delta	JSON parsing; TRY_CAST; quality rules; quarantine; keyed MERGE
04 Gold	Silver + operation logs → facts/dimensions	SHA-256 keys, groupBy daily aggregation, NOAA canonical views; overwrite/MERGE
05 BI	Gold Delta → Power BI	Databricks connector; Import-model refresh; DAX and relationships

Integration qualification: the scheduled GitHub collectors are source-defined; Databricks job JSON expresses dependencies but requires valid compute/paths, deployment and a successful end-to-end run. Import-mode Power BI refresh is a separate step; updating Gold alone does not guarantee the displayed report is current.

03 Gold dimensional model

A star / shallow-snowflake design: fact events and aggregates link to reusable descriptive dimensions.

The source DBML and diagram identify the target fact/dimension relationship structure. Surrogate hash keys are logical constraints maintained by transformations; Delta Lake does not automatically enforce them as database PK/FK constraints.

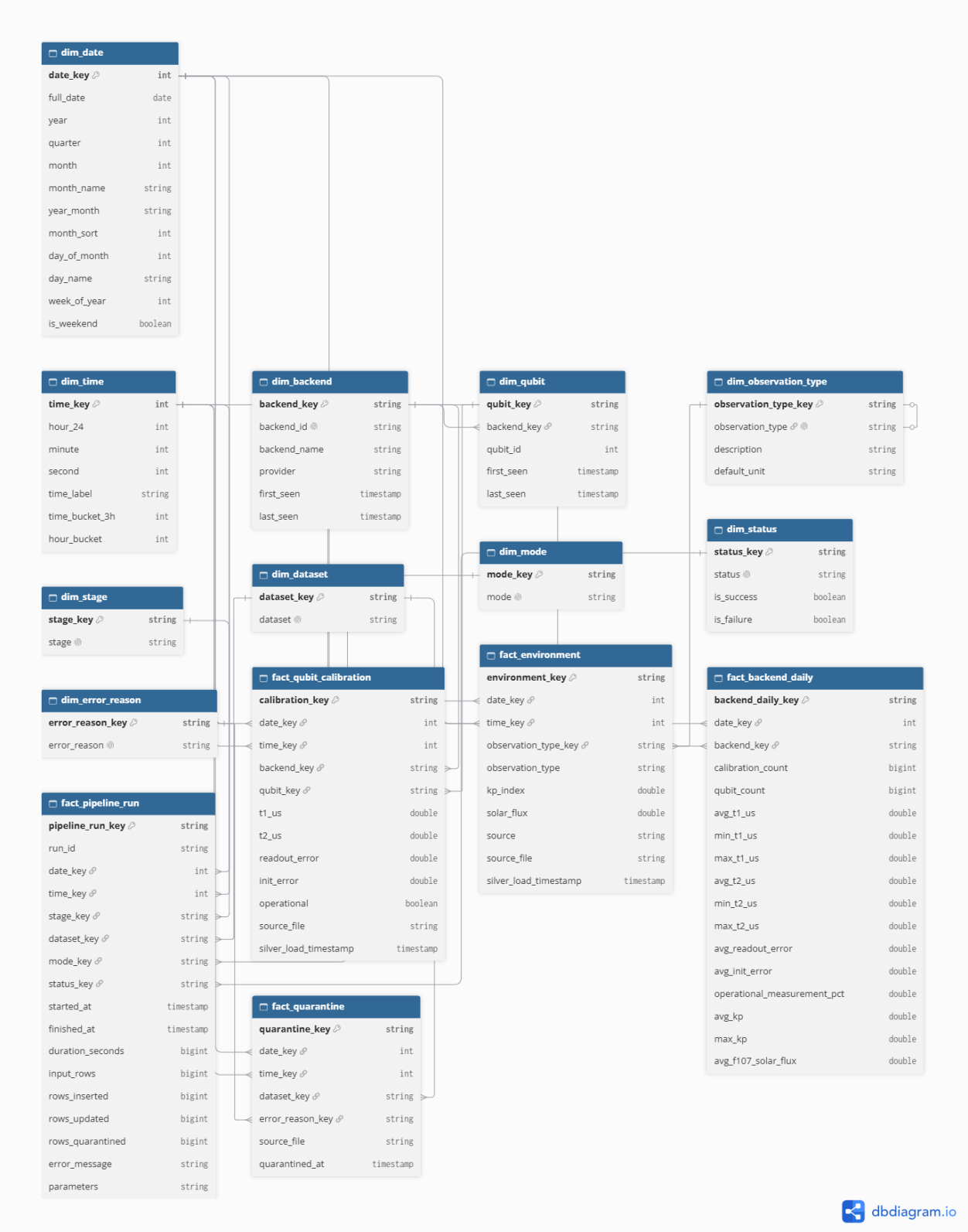


Figure 1. Gold fact/dimension ER diagram, drawn from docs/diagrams/quantalytics_phase3_gold_model.png. Model is conceptual and key integrity is implemented in code.

04 Gold data dictionary — fact tables

Five business-ready facts, including both atomic records and query-optimised daily summaries.

Fact / grain	PK / foreign keys	Measures and role
fact_qubit_calibration One qubit per calibration time	PK calibration_key FK date_key, time_key, backend_key, qubit_key	t1_us, t2_us, readout_error, init_error, operational; qubit-level reliability and distributions
fact_environment One canonical observation	PK environment_key FK date_key, time_key, observation_type_key	kp_index, solar_flux, observation_type; NOAA historical Kp and daily F10.7
fact_backend_daily One processor per day	PK backend_daily_key FK date_key, backend_key	calibration_count, qubit_count, avg/min/max T1/T2, mean errors, operational %, avg/max Kp, avg F10.7
fact_pipeline_run One logged execution event	PK pipeline_run_key FK date/time/stage/dataset/mode/status keys	duration_seconds, input_rows, rows_inserted, rows_updated, rows_quarantined; operational monitoring
fact_quarantine One rejected-source record	PK quarantine_key FK date/time/dataset/error_reason keys	source_file, quarantined_at; invalid-record investigations and error Pareto

Surrogate-key design

For descriptive entities (backend, qubit, observation type, log categories), SHA-256-derived string keys provide reproducible lookups. Date and time use integer keys (**yyyyMMdd** and **HHmmss**) for convenient BI joins. Facts use source record_key where appropriate or a hash of their grain, e.g. backend_id + date_key for daily aggregates.

The daily fact intentionally stores both IBM performance measures and joined day-level NOAA context, reducing repeated expensive aggregations in dashboard queries.

05 Gold data dictionary — dimensions

Dimension roles and exact key relationships used by the facts and Power BI model.

Dimension / primary key	Description / key relationships
dim_date | date_key (INT)	Full date, year, quarter, month, weekday/weekend; links every fact's date_key.
dim_time | time_key (INT)	Hour, minute, second, three-hour bucket; links atomic and operational time keys.
dim_backend | backend_key (STRING)	IBM processor code/name, provider, first/last seen; links daily fact and dim_qubit.
dim_qubit | qubit_key (STRING)	Per-backend qubit identifier, first/last seen; backend_key FK → dim_backend.
dim_observation_type | observation_type_key	Kp versus F10.7, description and unit; FK from fact_environment.
dim_stage | stage_key	Raw-to-Bronze, Bronze-to-Silver, Silver-to-Gold; FK from fact_pipeline_run.
dim_dataset | dataset_key	IBM, NOAA, Gold and logged datasets; FK from pipeline/quarantine facts.
dim_mode | mode_key	Full / incremental / backfill labels; FK from fact_pipeline_run.
dim_status | status_key	SUCCESS / FAILURE, success/failure flags; FK from fact_pipeline_run.
dim_error_reason | error_reason_key	Invalid-data failure labels; FK from fact_quarantine.

Semantic layer and BI joins

The Power BI project contains **19 manually specified core fact/dimension relationships**, including **dim_qubit.backend_key → dim_backend.backend_key**. Its generated local date relationships are additional Power BI auto-date artefacts, not Gold schema foreign keys. Cardinality is designed as many-to-one from facts into dimensions with single-direction filtering.

> A natural correctness check is to compare each fact FK against its dimension PK and inspect Power BI Model view for active relationships and ambiguous filter paths. Those live checks are not captured in the repository.

06 Pipeline integration and incremental design

How the Gold notebook is intended to work after Bronze and Silver processing.

Mode / component	Actual repository implementation	Implication
Gold full	MODE=full → fact writer uses Delta overwrite + overwriteSchema	Rebuilds all five fact tables from source DataFrames.
Gold incremental	If no supplied start date: last successful Gold log date minus 2 days; END_DATE defaults to today	Replays a rolling overlap to handle recent corrections.
Gold incremental writes	Delta MERGE per fact key; dimensions are MERGEd from distinct source entities	Existing rows are upserted rather than appended blindly.
NOAA canonicalisation	vw_noaa_kp_canonical selects noaa_full_% historical Kp; vw_noaa_f107_daily selects one reading per calendar day	A documented stopgap, not a fully normalised continuous live Kp series.
Gold audit	Writes shared pipeline_execution_logs stage=Silver-to-Gold, dataset=gold, mode, timing and JSON details	Details include source row counts and intended fact-row outputs.
Jobs	Separate IBM/NOAA JSON tasks: Bronze → Silver → Gold	Dependency graph defined; deployment and scheduling unverified.

Implementation cautions discovered by code review

- **Daily environmental freshness:** Gold Cell 7 reads **workspace.gold.fact_environment** before writing the newly built **fact_environment** in Cell 8. A first run can fail, and later runs can join stale NOAA data. Use the current **fact_environment** DataFrame for the daily joins.
- **Partial incremental filtering:** the environmental fact is constructed from unwindowed canonical views, so the input window does not constrain that fact the same way it constrains IBM source. This is an efficiency/semantics limitation.
- **Orchestration artefacts:** job specifications show **existing_cluster_id: ""**. Correct compute configuration and actual job/run evidence must be supplied before reporting fully automated successful production execution.
- **Gold log metrics:** the logger puts a total candidate fact-row count into rows_inserted and zero into rows_updated; these fields are not true Delta MERGE metrics for incremental Gold loads.

07 Decision log and data governance

Engineering choices, trade-offs and limitations defended against the Phase 3 marking criteria.

Decision	Rationale and trade-off	Evidence / qualification
Medallion + Delta	Raw replay is separate from typed validation and BI aggregations; Delta MERGE supports upsert and transaction history.	Collector/bronze/silver/gold notebooks.
Explicit schemas	Preserve source strings in Bronze; only validated numeric/timestamp/Boolean values enter Silver.	Bronze StructType, IBM from_json and TRY_CAST.
Quarantine rather than auto-evolve	Unknown IBM fields and invalid types are labelled and rejected, with other valid rows continuing.	Silver validation and QA notebooks; no mergeSchema claim.
Logical keys + MERGE	Deduplicate business grains before upsert; do not assume Delta-enforced PK/FK.	record_key + payload hash; row_number; Gold key MERGEs.
No Hive date partitions	Gold volume ~0.5M IBM observations: per-day directories risk excessive small files.	No partitionBy found; ZORDER calls per fact.
OPTIMIZE / ZORDER	Date + entity keys chosen for filter locality on analytical trends/joins.	Gold Cell 8 invokes OPTIMIZE ... ZORDER. No successful execution log saved in ZIP.
PII minimisation	Data represents public machine telemetry/space-weather readings, not people. API tokens are sourced from environment/secrets.	Collectors and transformed schemas; secrets are not stored as literal values.
Import-mode Power BI	Fast user interactions on moderate-sized Gold facts; periodic refresh required for freshness.	TMDL/Power Query connector definitions.

Privacy boundary: no PII fields have been identified in the Gold fact/dimension contracts. This conclusion applies to the inspected project data model, not to every possible future upstream feed. Credential governance still matters.

08 Power BI implementation and analytical questions

A model-first dashboard design that uses Gold facts rather than Silver tables.

Research question	Gold data and intended BI visual
BQ1: QPU trends	fact_backend_daily with dim_date/backend → daily average T1/T2 time series and processor KPI comparison.
BQ2: Reliability distribution	fact_qubit_calibration with dim_qubit/backend → T1/T2 scatter, variation, readout error and qubit-level tables.
BQ3: Space-weather association	fact_environment and fact_backend_daily → Kp/F10.7 context, scatter correlations and storm/quiet comparison. This is exploratory, not causal.
BQ4: Pipeline health	fact_pipeline_run and fact_quarantine → success, execution times, rows ingested, error types and failure trends.

BI artefact observed in the repository

The native Power BI project at **dashboard/Quantalytics_pbip_v2/Quantalytics/** uses the Databricks connector with configurable host, SQL warehouse path, catalog **workspace** and schema **gold**. It specifies Import storage mode, editable measures and a model-backed report. The README lists five core pages and seven advanced pages added in v2.

Core pages	Added analysis pages
QPU Health Overview	Exec Insights; Storm Impact
Backend & Qubit Reliability	Lag & Sensitivity; Solar Regimes
Space Weather vs QPUs	Rhythms; Coherence Physics
Pipeline & Data Quality	Coverage & Trust
About & Data Notes	-

Observed data examples (team-shared CSV exports, 24 Oct 2026)

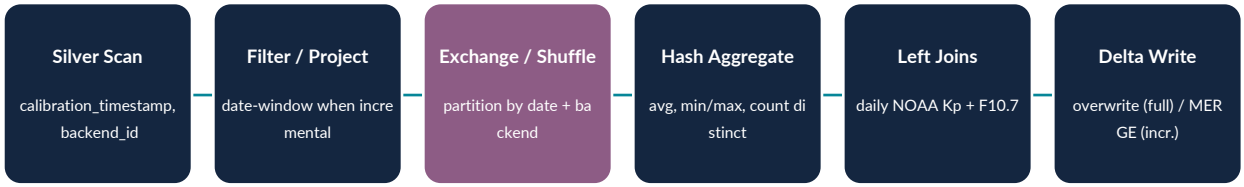
665 backend-day rows; **1,000** exported qubit fact rows; **1,000** exported NOAA fact rows; **18** exported execution events; **0** rows in that quarantine export. The qubit and NOAA exports were capped at 1,000; do not describe these as full Gold table totals. Earlier user-supplied Power BI screenshots demonstrate four populated report pages but do not verify the latest v2 twelve-page project in Power BI Desktop.

> **Refresh nuance:** The report is Databricks-connected by source definition. With Import mode, Gold data changes appear only after Power BI refresh; this is not continuously live querying.

09 Spark UI DAG and physical execution

Required execution analysis of the Gold daily backend aggregation.

Selected transformation: **fact_backend_daily** construction in the Gold notebook, involving IBM grouping/aggregates and two left joins to day-level NOAA Kp and F10.7. The following is a conceptual reconstruction from the code, **not an actual Spark UI capture**.



Conceptual plan reconstructed from notebook transformations; not a Spark UI screenshot.

Actual plan may contain adaptive exchanges, broadcast joins, and partial/final aggregations.

What Spark is expected to do

- **Scan and filters:** Spark reads Delta Silver rows and applies the selected event-date window in incremental mode. Partition/file pruning and predicate pushdown depend on the physical plan and data layout; they are not proven by code alone.
- **Shuffle:** Grouping by date and backend, particularly countDistinct, usually causes an Exchange between partial and final aggregate stages. Physical strategy depends on runtime statistics and adaptive execution.
- **Joins:** Daily Kp/F10.7 aggregates join on date_key. Spark may broadcast the smaller NOAA day aggregates or shuffle both sides; verify operator names in the real UI.
- **Write:** Full mode overwrites facts; incremental mode performs Delta MERGE keyed by fact grain. ZORDER optimisation adds its own subsequent work.

MANDATORY EVIDENCE STILL REQUIRED: Insert a genuine Spark UI DAG / SQL execution screenshot here before submission. Open the Databricks Gold run, inspect Spark UI / SQL query execution for the fact_backend_daily query, capture the plan and record operator names, shuffle read/write bytes, tasks and actual runtime. The repository has no such screenshot.

10 FinOps — observed time and cost model

Quantify what the available evidence shows, then separate assumptions from actual billing.

Source evidence: the team-shared **fact_pipeline_run** CSV export contains 18 pipeline events from 11–21 Oct 2026. These are logged per-stage runtimes, not whole-workflow durations. They do not include a verified Silver-to-Gold run in that exported dataset.

EVENTS	MEAN DURATION	MEDIAN DURATION	OBSERVED RANGE
18	**15.7s**	**12.5s**	**0–36s**
Exported operational fact row	Across stage events	Across stage events	Not workflow wall-clock

Measurement limitations

No **DBU consumption, Azure VM bill, cluster startup time, SQL warehouse runtime, egress or storage billing** was supplied in the repository. Therefore an actual cost per incremental execution cannot be stated reliably. Databricks recommends benchmarking representative workloads and examining billed usage data rather than inferring cost from notebook wall-clock time alone.

Illustrative enterprise annualisation — transparent assumptions

For an hourly pipeline: **8,760 runs/year**. Assume 2 billable DBU per active compute hour and a reference Azure Jobs Compute rate of **USD 0.30/DBU-hour** (published pricing reference, region/SKU-dependent). This is a DBU-charge-only illustration, **not** the observed student-workspace cost and **excludes** Azure VM/infrastructure, storage, networking, Power BI licensing and idle time.

Assumed active compute per run	DBU/run at 2 DBU/h	Indicative DBU/year	Indicative USD/year
5 min	0.167	1,460	$438
10 min	0.333	2,920	$876
20 min	0.667	5,840	$1,752

Calculation: annual DBU-only cost = 8,760 × (active minutes / 60) × DBU per active hour × USD per DBU-hour. Example for 10 minutes: 8,760 × (10/60) × 2 × 0.30 = **USD 876/year**.

Optimisation plan

- Use change-data windows rather than rereading entire Silver history each hour; reconcile late-arriving records with a bounded overlap.
- Reduce repeated full-fact OPTIMIZE jobs; benchmark write frequency, ZORDER value and file sizes.
- Cache only reused inputs during one workflow, and avoid repeated count() actions where practical.
- Right-size on-demand/serverless compute, terminate idle warehouses, batch low-volume ingestion and monitor job startup overhead.
- Track actual DBU via billing exports/system.billing.usage (when the workspace exposes it); allocate by job and compare cost per ingested row.

External reference: Azure Databricks pricing; Databricks usage/cost guidance.

11 Risks, acceptance tests and submission status

Prioritised remediation derived from inspected code, source files and available run evidence.

Priority	Risk / evidence gap	Recommended close-out test
HIGH	No real Spark UI DAG screenshot in ZIP.	Run Gold aggregation; capture SQL / Spark UI DAG, stages/shuffles, and operator names.
HIGH	Gold daily NOAA join reads stored fact_environment before current write.	Change to freshly computed fact_environment DataFrame; verify new Kp/F10.7 visible in same run.
HIGH	Job JSON has empty existing_cluster_id; deployment unverified.	Configure compute, deploy both jobs, run fresh incremental Bronze → Silver → Gold, save run task graph.
HIGH	No Gold notebook execution outputs and no Gold-stage logs in supplied 18-row export.	Export latest Silver-to-Gold SUCCESS/FAILURE logs and fact counts, rerun to show idempotency.
MED	Gold incremental environment view is not event-window filtered.	Scope canonical NOAA facts to intended window or document controlled full reprocessing.
MED	Gold audit rows_inserted does not represent actual MERGE inserts.	Obtain real Delta operationMetrics or relabel written/aggregated-row metric accurately.
MED	NOAA historical Kp filtering is a deliberate stopgap and may omit gaps/live records.	Validate coverage vs source, normalise 3-hourly series, document gaps.
MED	Power BI Import model requires explicit refresh.	Show model relationships and refresh a newly changed Gold date; export/publish the dashboard.
MED	Actual DBU and cloud credit use not provided.	Add jobs usage screenshot/export and measured compute cost per typical incremental run.

Submission readiness: Gold code, ER diagram and Power BI project source are in the ZIP. A fully defensible final submission still needs live Spark DAG evidence, measured billing or a documented inability to obtain it, verification of deployed job dependencies, and a refresh-tested .pbix or published dashboard.

12 Evidence map and references

Precise traceability from report statements to paths in the supplied repository snapshot.

Claim / artefact	Repository file or shared evidence
Phase 3 specification	Phase3/phase3.txt
Gold notebook: dimensions, facts, writer, audit, ZORDER	spark/gold/1. Gold Pipeline.ipynb (notebook Cells 2–9)
Gold schema/keys, 5 facts and 10 dimensions	docs/diagrams/quantalytics_phase3_gold_model.dbml and .png
Design rationale, PII and current limitations	Phase3/Gold_Layer_Explanation.md; Phase3/Phase3_Implementation_Guide.md
Bronze and Silver lineage, quality and logging	spark/bronze/ibm_bronze_load.ipynb; spark/bronze/noaa_bronze_load.ipynb; spark/silver/1. Silver Pipeline.ipynb
Gold orchestration definitions	databricks/jobs/quantalytics_ibm_job.json; databricks/jobs/quantalytics_noaa_job.json
Scheduled acquisition	.github/workflows/ibm-collect.yml; .github/workflows/noaa-collect.yml
Power BI model and report	dashboard/Quantalytics_pbip_v2/Quantalytics/Quantalytics.pbip; ...SemanticModel/definition/relationships.tmdl; project README
Observed demo counts and timings (supplementary uploads)	Table_Data.csv; Table_Data (1).csv; Table_Data (2).csv; Table_Data (3).csv; Table_Data (4).csv shared separately
Actual DAG screenshot and billed usage	Not found in attached ZIP; must be supplied from Databricks.

Technical references

- Quantalytics GitHub repository (snapshot from uploaded ZIP, commit 78b8d4d, not a fresh remote verification).
- Microsoft Azure Databricks pricing (pricing varies by region, cluster, SKU and date).
- Databricks serverless billing and usage guidance (benchmark representative workloads).
- Microsoft Learn: Databricks Power Query connector (Import / DirectQuery connection capabilities).
- Microsoft Learn: Power BI semantic models (Import data needs refresh).

> Report produced 25 October 2026. All quantified actuals and implementation claims are scoped to the supplied ZIP and expressly identified supplemental exports. Unverified claims are labelled rather than presented as completed tests.